allback

ViSongSearch: Finding Vietnamese Songs from Imperfectly Remembered Lyrics

[Authors: names, FPT University, Vietnam; corresponding e-mail]

Abstract

People who look for a song whose title they have forgotten type what they remember of it: a line of the lyrics, often without diacritics, misheard or loosely paraphrased, or a description of the song. In Vietnamese, a tonal language written with diacritics and with strong dialectal variation, these imperfections change the written form of many syllables, and no test collection measures how search engines cope with them. We present ViSongSearch, a known-item test collection of 6337 Vietnamese songs grouped into clusters of versions, with song-disjoint splits and five query types: exact lyric fragments, the same fragments without diacritics, fragments passed through a noise channel for dialectal mergers, tone confusions, input-method residue and keyboard slips, and paraphrases and descriptions written by a large language model under leakage filters. As no human queries are available, we test the conclusions with a noise channel re-estimated from independent transcriptions of the same songs, agreement between system orderings and discriminative power. The index unit and the handling of diacritics matter most: syllable bigrams over unmarked or restored text reach MRR@10 of 0.929 or more on exact, unmarked and noisy fragments and keep 0.711 when half of the syllables are corrupted, while character n-grams are the weakest analyzer. Paraphrases defeat lexical matching (MRR@10 at most 0.354); zero-shot bi-encoders do better on them ([?]) but need the diacritics restored before encoding. Noise-aware fine-tuning improves a small bi-encoder mostly on fragments that lexical retrieval already solves. No single fusion weight lets a dense model help, because every weight that helps paraphrases hurts fragments; weights chosen per predicted query type raise paraphrases from 0.344 to 0.532 at no cost elsewhere, and cross-encoder reranking adds [?] on paraphrases at several seconds per query on a CPU.

Keywords: known-item search; lyrics retrieval; Vietnamese; diacritic restoration; query noise; test collection; dense retrieval; rank fusion

1 Introduction

A common search need is a song one can hum but cannot name. Song search services, video platforms and web search engines receive such requests as short queries that quote whatever the person remembers, and community question-answering sites collect longer ones: among solved music tip-of-the-tongue requests, lyric quotes and descriptions of what the lyrics are about are frequent cues, next to the genre and the circumstances in which the song was heard [9]. The remembered words are often wrong. Misheard lyrics are common enough to have their own retrieval literature, which matches queries to lyrics by phonetic similarity [21, 41].

Vietnamese makes the problem harder in three ways. First, the orthography marks every syllable with vowel diacritics (*ă â ê ô ơ ư đ*) and one of six tones, five of them written as marks (*má mà mả mã mạ*). Many people type without marks, because it is faster or because the input method is off, so the query *bo* must be matched to one of *bờ, bố, bỏ, bộ, bó, bổ, bơ, bô* and more: the lyrics of our catalogue alone contain 17 marked forms of *bo*, and an unmarked syllable in running lyrics could stand for 8.0 marked forms on average ([?] per distinct unmarked syllable: the frequent syllables have the most forms). When a Telex input method is switched off halfway, the keystrokes themselves appear in the query (*nhuwngx* for *nhữ̃ng*). Second, the major dialects

merge sounds that the spelling keeps apart: Hanoi speech does not distinguish the onsets *tr/ch*, *s/x* and *d/gi/r*; Southern speech merges onsets *v/d/gi*, the codas *n/ng* and *t/c*, and the tones *hỏi* and *ngã*; *l/n* are confused in parts of the North [23, 26]. A listener who writes down a lyric that they have only heard therefore produces systematic misspellings. Third, words are written as space-separated syllables, so a search engine must decide whether to index syllables, segmented words or character sequences, and each choice reacts differently to the errors above.

Retrieval research on Vietnamese has produced embedding benchmarks [33, 37], rerankers [18] and diacritic restoration systems [19, 38], but the queries in these benchmarks are well-formed questions, and restoration is evaluated on running text rather than on short, out-of-context queries. Work on the robustness of neural retrievers to typos [36, 45, 53, 54] concerns English character-level errors. To our knowledge there is no test collection for Vietnamese lyrics search, and no study of how lexical, dense and hybrid retrieval cope with the ways Vietnamese users actually misremember and mistype lyrics.

This paper makes four contributions:

1. **A known-item test collection** for Vietnamese song search (Section 3): the lyrics of 6337 songs, [?]% of them from a uniform random sample of the largest Vietnamese chord site, grouped into clusters of versions so that any version of the wanted song counts as correct, with song-disjoint splits; five synthetic query types that range from exact lyric fragments to descriptions of the song, the last two also typed without marks; three test-only variants that change how the noisy, paraphrased and described queries were made; and a noise sweep under three channels.

2. **Two noise channels for Vietnamese queries**: one that models dialectal onset and coda mergers, tone confusions, input-method residue and keyboard slips at the level of syllable structure (Section 3.5), and one sampled from the substitutions observed between independent transcriptions of the same songs (Section 3.7).

3. **A comparison of retrieval methods** under these conditions (Sections 4 and 6): lexical retrieval with nine analyzers, query likelihood and pseudo-relevance feedback, diacritic restoration with a Kneser–Ney language model, metadata fields, dense bi-encoders used zero-shot and after noise-aware fine-tuning, each on the queries as typed and after diacritic restoration, several ways of fusing lexical and dense lists, among them weights that depend on a predicted query type, and cross-encoder reranking, all tuned on a development split and tested with paired randomization tests.

4. **Validity checks that need no participants** (Sections 3.7 and 6.6): the realism of the channels against real transcriptions, the sensitivity of the conclusions to the prompt and to what the query writer saw, agreement between system orderings, the sensitivity of each query type, and results outside the catalogue on which the method was developed. The code, the queries, the run files and per-song checksums are released [Authors: repository URL].

Our main findings are these. (i) The index unit and the handling of diacritics decide most of the outcome: syllable bigrams over restored text reach MRR@10 0.999, 0.996 and 0.933 on exact, unmarked and noisy fragments, while character trigrams, segmented words and pseudo-relevance feedback do not help. (ii) Paraphrases are the open problem: the best lexical model reaches 0.354 on them and a zero-shot bi-encoder [?]. (iii) Bi-encoders need the diacritics back: restoring them before encoding raises mE5-base from [?] to [?] on unmarked fragments. (iv) Noise-aware fine-tuning makes a small bi-encoder better on fragments and noisy queries, which lexical retrieval already solves, and adds nothing to a hybrid. (v) A dense list helps a hybrid only with weights that depend on the query type; the best single weight on development queries is zero. (vi) The ordering of the systems is stable under a noise channel estimated from real transcriptions and across query types, except on paraphrases, which order the systems differently from every other type.

2 Related work

Known-item search and synthetic queries. In known-item search the user wants one particular document they have seen before [35]. Because each query has a single answer, known-item test collections can be built by simulating queries from the target document: Azzopardi et al. [7] sampled query terms from known documents in six European languages and compared the resulting system rankings with those obtained from real queries. Large language models now generate training queries for retrieval models [11] and query variants or whole test collections for evaluation [3, 39]. We follow both lines: rule-based simulation for lyric fragments, where the generative process is clear, and an LLM for paraphrases and descriptions, with filters against leakage of the answer. Without real queries we test how far conclusions depend on the generator with the reliability analyses used for test collections: agreement between system orderings [48] and discriminative power [44].

Lyrics search and tip-of-the-tongue retrieval. Ring and Uitdenbogerd [41] studied the misheard lyric matching problem and matched English queries to lyrics with phonetic string similarity; Hirjee and Brown [21] learned a probabilistic model of phoneme confusions from real misheard lyrics, and Xu et al. [51] made lyric search robust to misheard queries with a phonetic confusion matrix. Tip-of-the-tongue (ToT) retrieval addresses complex known-item requests written by people who cannot recall an identifier: Arguello et al. [5] analysed movie requests, Bhargav et al. [8] released a known-item dataset built from such requests, the TREC ToT track has run since 2023 [6], and Bhargav et al. [9] studied music ToT requests, where lyric quotes and lyric descriptions are common cues. For Vietnamese songs, recent work addresses automatic lyrics transcription from audio [31], not retrieval from remembered text.

Robustness to query variation. Penha et al. [36] generated query variations, including typos, to probe the robustness of retrieval pipelines. Zhuang and Zuccon [53] trained BERT-based retrievers and rankers with typo-augmented queries, and Zhuang and Zuccon [54] combined a character-level encoder with self-teaching; Sidiropoulos and Kanoulas [45] analysed dual encoders under misspellings. These studies inject character edits into English queries. Vietnamese errors are better described at the level of the syllable's parts (onset, rhyme and tone) and of the input method, which is what our noise channel models.

Vietnamese text processing and retrieval. Diacritic restoration has been treated as machine translation [38] and with a hybrid of rules and a Transformer decoder [19]. Word segmentation tools such as VnCoreNLP [49] and pyvi [47] join syllables into words, and PhoBERT [30] is pre-trained on segmented text. Vietnamese retrieval is evaluated in VN-MTEB [37] and the VCS benchmark [33], and multilingual encoders such as mE5 [50] and BGE-M3 [14] cover Vietnamese. None of these resources contains lyrics or noisy queries.

Hybrid retrieval and query-dependent combination. Dense bi-encoders [25, 40] complement lexical models, and long documents are commonly scored by their best passage [17]. Reciprocal rank fusion (RRF) combines ranked lists without score normalisation [16]; Bruch et al. [13] found a convex combination of normalised scores more effective and more sample-efficient to tune than RRF. Cross-encoders rerank a short candidate list [34]. Choosing the retrieval strategy by the type of query is an old idea: Kang and Kim [24] classified web queries as topic relevance, homepage or service finding and combined content, link and URL evidence differently for each class, and Arabzadeh et al. [4] predicted from the query whether a dense or a sparse ranker would serve it better. We apply this idea to a hybrid of lexical and dense lists, where the query types differ in a way that a classifier can see (marks, length, wording), and we

compare type-dependent RRF weights with convex fusion, routing and a selector that needs no query type.

3 The ViSongSearch collection

3.1 Corpus

Most songs come from Hợp Âm Chuẩn (*hopamchuan.com*), the largest Vietnamese chord site, which we crawled for this study. The crawler visited song identifiers 1 to 90000 in a fixed random order, at one request per second and within the limits of the site's `robots.txt`, until it had collected 8234 song pages with lyrics (11002 identifiers visited in October 2026; identifiers without a song return an error page). Because the identifiers are visited in random order, these songs are a uniform sample of the site rather than of its most popular songs, and the sample does not depend on where the crawl stopped. Lyrics are read from the lyric spans of the page, so chord names never enter the text; the crawler also keeps the composer, the singers, the genre and the view count. The other 570 input songs come from a focused crawl of three music sites made earlier for a search-engine course project. After cleaning they form the *pilot catalogue* of [?] songs, on which the method was developed; they were chosen by a focused crawler, not at random. [?] of them are Hợp Âm Chuẩn pages that the uniform crawl fetched as well, and its copy is kept. Of the final catalogue, [?] songs ([?]%) therefore come from the uniform sample and [?] from the pilot crawl alone ([?] from Hợp Âm Chuẩn, 309 from *nhaccuatui.com* and 20 from *nhac.vn*); Section 7 repeats the main results without the pilot clusters.

Lyrics pages carry text that is not lyrics and that would leak the answer into the index or the queries: credits (composer, singer, *lời Việt*, beat and mix credits), page headers (*Đăng bởi…*, *Bài hát:…*), interface text (*Xem thêm*, *Nghệ sĩ*, *Theo dõi*) that is followed by the artist's name on one of the sites, and, on the chord site, notes on the key, the capo and the chord loop (*Tone chuẩn Eb. Capo 1*, *Vòng hợp âm…*) and lines of chord names. Cleaning removes these lines, stops at "show more" markers, maps homoglyphs (Greek and Cyrillic letters, decorative letters such as an *N* with a hook) to Vietnamese letters and normalises Unicode to NFC. Uploaders of chord sheets sometimes cut a syllable in two around a chord (*sươ [Am] ng*); two neighbouring tokens are joined when the joined form occurs in at least ten songs and one of the parts is at least twenty times rarer than it (1030 joins). Of 8804 songs we drop 38 with fewer than 40 syllables, 2202 in which fewer than 25% of the syllables carry a Vietnamese mark (mostly songs in English, and other languages written without marks, such as romanised Korean), 198 whose marks are not Vietnamese (Chinese in pinyin, recognised by tone marks that Vietnamese does not use, and lyrics in which almost no syllable carries a mark that only Vietnamese uses, such as Chinese typed with Vietnamese letters) and the 29 pages that both crawls fetched, keeping 6337. The median song has 189 syllables in 21 lines; the lyrics use 11613 distinct syllables, which collapse to 7199 without marks; 6056 songs have an artist (composer or performers) and 5055 a genre. For dense retrieval, each song is split into passages of four lines with a stride of two lines (79328 passages, 36 syllables on average), so that every span of up to three consecutive lines lies inside one passage.

3.2 Song clusters

The same song appears on several sites and in several versions (remixes, chord versions, covers), and for a known-item query any of them is a correct answer. We therefore evaluate over clusters of versions. Two songs join a cluster when the Jaccard similarity of their sets of five-syllable shingles of unmarked lyrics is at least 0.5, or when their normalised titles are equal (bracketed parts and tags such as *remix*, *live*, *beat* removed, marks stripped) and their Jaccard similarity is at least 0.05. Candidate pairs for the first rule come from locality sensitive hashing of MinHash signatures [12] with 128 permutations, of which 42 bands of 3 rows use 126; a pair at similarity

0.5 becomes a candidate with probability 0.996, and the similarity of each candidate pair is then computed exactly. The second rule merges versions whose lyrics were transcribed very differently, which reached a similarity of about 0.09 in a pilot study, while different songs that share a one-word title such as *Em* have a similarity of 0. Clusters are the connected components of these pairs. The collection has 6213 clusters; 93 of them contain more than one version (217 songs), and the largest has 9 versions. A ranking of songs is turned into a ranking of clusters by keeping the first song of each cluster, and the rank of the target cluster defines all metrics. To check the clusters, we inspected the [?] version pairs whose lyrics agree least (syllable-sequence similarity below 0.5, against a median of [?] over the [?] pairs inside clusters): [?] are remixes, covers or shorter transcriptions of one song, and one joins two different medleys with the generic title *Mashup nhạc trẻ*, which are never query targets (below). Among songs in different clusters, [?] pairs share both the normalised title and an artist; none of them has the same lyrics (two are different Vietnamese lyrics written to one foreign melody).

Medleys and mashups mix several songs, so a fragment of one has two correct answers. They stay in the index as distractors but are never query targets. A song is treated as a medley when its title says so (*liên khúc*, *mashup*, *medley*; 165 songs), or when its title lists several parts (*A x B*, *A - B + C*), one of which is the title of another cluster from which the song borrows at least 10% but less than 80% of its shingles (9 songs). Lyrics alone cannot separate a mashup from the songs it contains, since both share the same chorus; a song made almost entirely of another one is a version of it, hence the upper bound.

3.3 Splits

Clusters are assigned to training, development and test splits (70/10/20%) by hashing the most frequent normalised title of the cluster (its *key*), so that a song keeps its split when the catalogue grows. The collection has 4301/620/1292 clusters (4393/627/1317 songs). All songs stay in the index: the split only decides which songs serve as query targets. In the development and test splits, queries are made for one version per cluster, the one with the longest lyrics that is not a medley, so that every query in these splits is about a different song. Every learned component is trained on queries about training songs only, every parameter is tuned on development queries, and the test queries are used once per system.

3.4 Query types

Table 1 illustrates the query types; Table 3 gives their statistics.

Exact fragments (Q1). People recall short stretches of lyrics, often the hook. A fragment is a run of 5 to 12 consecutive syllables (length uniform) that starts at a random line and may cross a line break. A fragment that also occurs verbatim (after removing marks) in a song of another cluster is rejected, because such a query has more than one correct answer; the check tests every song that contains all syllables of the fragment. It rejected [?] of the fragments drawn ([?]%). Shorter fragments would often have no single answer: of random fragments of test songs, [?]% of those with three syllables and [?]% of those with four also occur in another cluster, against [?]% with five.

Unmarked fragments (Q2). The fragment with all marks removed. In 15% of the queries, one or two syllables are instead written as their Telex key sequence (*những* → *nhuwngx*, *vắng* → *vawngs*), as when the input method is turned off while typing.

Misheard or mistyped fragments (Q3). The fragment with 10, 20, 30 or 40% of its syllables (at least one) corrupted by the noise channel of Section 3.5. On the test split, 26% of the syllables of a Q3 query are changed on average.

Table 1: Query types, illustrated on an invented lyric line ('the rainy afternoons on the empty street'). Changed syllables in bold.

Type	Construction	Example
Q1	exact fragment of 5–12 syllables	*những chiều mưa rơi trên phố vắng*
Q2	Q1 without marks; 15% keep 1–2 Telex syllables	***nhuwngx*** *chieu mua roi tren pho* ***vawng***
Q3	Q1 through the noise channel (10–40%)	***nhửng*** *chiều mưa* ***dơi*** *trên phố* ***vắn***
Q4	LLM paraphrase of a fragment	*buổi chiều trời mưa con đường không có ai*
Q5	LLM description, title forbidden	*bài hát buồn về người đi một mình dưới mưa nhớ người yêu cũ*
Q4u	Q4 without marks	*buoi chieu troi mua con duong khong co ai*
Q5u	Q5 without marks	*bai hat buon ve nguoi di mot minh duoi mua nho nguoi yeu cu*
Test-only variants		
Q3u	Q3 without marks	***nhung*** *chieu mua* ***doi*** *tren pho* ***van***
Q4s	LLM paraphrase, second prompt (5–12 syllables)	*chiều mưa trên phố không người*
Q5g	LLM description written from a gist, without lyrics, title or artist	*tìm bài hát buồn về người nhớ người yêu cũ trong những chiều mưa*

Paraphrases (Q4) and descriptions (Q5). A listener may remember what a line says but not its words, or remember only what the song is about. Both kinds of query are written by an LLM (Section 3.6).

Unmarked paraphrases and descriptions (Q4u, Q5u). The LLM writes with marks, but people who type without marks do so for every kind of query. Q4u and Q5u are the Q4 and Q5 queries with every mark removed. They show whether a system's effectiveness on paraphrases and descriptions survives the loss of marks. We report them separately and keep them out of the macro average, which stays over the five main types.

Test-only variants. Three variants probe how much the conclusions depend on how the queries were made; they exist for test songs only and never enter training or tuning. *Q3u* types the Q3 queries without marks (15% with Telex residue, as Q2). *Q4s* asks the LLM, with a second prompt, for a short and loose recollection of the same fragment as Q4 (5–12 syllables instead of 8–20). *Q5g* removes what the Q5 writer could copy: one LLM call summarises about ten lines of the lyrics in one sentence of its own words (the *gist*, without the title), and a separate call, which sees only the gist, writes the query (Section 3.6).

Noise sweep. Each test fragment of type Q1 is also corrupted at 0, 10, 20, 30, 40 and 50% (1241 fragments, 7446 queries), which traces how quickly each system degrades. Two further sweeps repeat the levels from 10 to 50% on the same fragments with channels derived from real transcriptions (Section 3.7): W samples the substitutions observed between transcriptions, and E re-weights the operations of the hand-set channel by the observed shares.

Each training song yields two fragments, and the representative of each development and test cluster one; each fragment gives one query of types Q1, Q2 and Q3. Q4 and Q5 exist for a sample of the development and test clusters (one of each per cluster, 604 queries; Section 3.6), and so do their unmarked versions.

Table 2: Operations of the noise channel. Shares: operations applied to the test queries of type Q3.

Operation	Rule	Examples	Share
Onset merger	*s*↔*x*, *tr*↔*ch*, *l*↔*n*, *d*→{*gi*, *r*, *v*}, *gi*→{*d*, *r*}, *r*→{*d*, *gi*}, *v*→*d*	*sương*→*xương*, *trời*→*chời*, *về*→*dề*	15%
Coda merger	*n*↔*ng*, *t*↔*c*	*buồn*→*buồng*, *mắt*→*mắc*	10%
Tone	*hỏi*↔*ngã*; other tones dropped with $p = 0.5$	*những*→*nhửng*, *lòng*→*long*	17%
Keyboard slip	adjacent-key substitution, deletion, doubling, transposition	*chiều*→*ciều*, *chhiều*	59%

3.5 A noise channel for Vietnamese queries

A Vietnamese syllable consists of an optional onset, a rhyme (medial, nucleus, coda) and a tone. The channel corrupts a syllable with one of four operations, chosen in an order sampled with weights 0.30 (onset), 0.15 (coda), 0.25 (tone) and 0.30 (keyboard); the first operation that applies to the syllable is used. Table 2 lists them.

The onset and coda operations follow the dialectal mergers described in the introduction [23, 26]; the *hỏi/ngã* swap reflects the merger of these tones in Southern and Central speech; dropping a tone mark is a typing slip. Keyboard slips act on unmarked letters, using QWERTY neighbours. Many syllables admit no onset or coda confusion (for example those with the onsets *b, m, h* or open rhymes), and keyboard slips apply almost everywhere, so they make up most of the applied operations even though their weight is not the largest (Table 11). The channel is deliberately simple and its weights were set by hand; Section 3.7 compares the substitutions it makes with those observed between real transcriptions and repeats the sweep with two channels derived from them.

3.6 LLM-written queries

Q4 and Q5 queries were written by an LLM (Claude, Anthropic) in October and November 2026, from the instructions given in Appendix A; each query was written from its own instructions and item, and the items were processed in batches. For Q4 the item is a lyric fragment: two consecutive lines of the song, drawn at random (with the song as seed) among the pairs of lines that have at least ten syllables, at least 60% of them distinct, and mostly Vietnamese words; the writer rewrites it as a listener who remembers its meaning but not its words. For Q5 the item is the title (which must not be written), the composer, the singers and the first sixteen lines of the lyrics, to be turned into the query of someone who has forgotten the title. Queries are written in lowercase, with marks. Writing them for every development and test cluster was beyond our budget, so they exist for every development and test cluster of the pilot catalogue and, among the clusters our crawl added, for a random sample: a cluster is sampled when a hash of its key, independent of the split hash, falls below 0.12 for test clusters and 0.06 for development clusters. Like the split, the sample does not depend on the order in which songs were crawled. This gives 604 queries for 75 development and 227 test clusters (248 written for the pilot clusters, 356 for the sample). Every query passes automatic filters on its cleaned, unmarked form: at least four syllables; for Q5, no occurrence of the title when the title has two syllables or more (one-syllable titles such as *Mưa* are ordinary words that a description cannot avoid); a longest run copied from the lyrics, which may cross a line break, shorter than five (Q4) or six (Q5) syllables; and for Q4, at most 70% of its syllables taken from the source fragment. The copy filter is thus looser than the instruction, which forbids four copied syllables in a row. A query that failed a filter was returned to the writer with the reason and rewritten. On the test split, Q4 queries have 17.2 syllables on average and their longest run copied from the lyrics is 2.2 syllables; 0.70

of their syllables occur in the target song, against 0.41 for a random other song (Table 3). The pilot descriptions followed a template (most named the singer or composer and began with *bài*, 'the song'), so the descriptions of the sampled clusters were written in more varied forms. Of all test Q5 queries, 49% name the singer or composer, and 64% begin with the most common first word, *bài*. Because their writer saw the lyrics, 0.72 of their syllables occur in the target song.

Variants. The variants address the two most obvious artefacts of this setup: paraphrases longer than a remembered line (17.2 syllables) and descriptions written with the lyrics and the artist in view. Q4s uses the Q4 items with a prompt that asks for 5–12 syllables (Appendix A); a Q4s query must pass the Q4 filters and have 4–14 syllables, and the [?] of [?] that failed were left out rather than rewritten, leaving 199 (10.7 syllables on average). For Q5g, a first session wrote a gist of each test song from about ten lines of its lyrics, without title or artist, and was told not to reuse three consecutive syllables of them; a second session, which had access to the gists only, wrote the queries. Q5g queries pass the Q5 filters (224 of [?] kept). Both variants come from the same model family as Q4 and Q5; we had no access to a second family, so the variants test the sensitivity to the prompt and to the information given to the writer, not to the generator.

3.7 Validity checks without participants

Synthetic queries may favour some systems, and a single generator may decide the conclusions. Without human queries we cannot measure validity directly. We check the noise model against text written by people, test how far the conclusions depend on the way the queries were made, and measure the reliability of each query type.

Variation between transcriptions. Versions of a song in one cluster were posted by different users or come from different sites, so they are transcriptions of the same lyrics by people. We align the syllables of each pair of versions (at most three pairs per cluster; pairs whose syllable sequences match less than half are medleys or different songs and are skipped) and class every substituted syllable as a tone change, another vowel mark, an initial consonant change that keeps the rest of the syllable (one of the dialectal mergers of the channel, or another initial), a final consonant change, a one-letter difference, or a different word. Spelling conventions (the tone placed on another vowel, as in *hoá/hóa*, or *i* for *y*) and tokens that are not Vietnamese syllables are not counted. Intervals are 95% bootstrap intervals over pairs. Two versions on one site may descend from the same transcription, so we also report the pairs whose versions come from different sites. These transcriptions show how people write down lyrics they know, at a rate of about one substitution per hundred syllables, not how they recall lyrics they half remember: they test the kinds of error the channel produces, not its rates.

Channels derived from the transcriptions. The *observed* channel W draws a kind of substitution with its observed share and applies it to a random syllable it fits: a different word (one that replaced this syllable in some transcription, or else one of the syllables seen in whole-word substitutions), a tone change with the observed transitions between tones, an initial change (a listed merger, or an initial drawn from the observed substitutions and kept only if the spelling stays valid), a final change, another vowel mark, or a keyboard slip. The *re-weighted* channel E keeps the four operations of the hand-set channel and sets their weights to the observed shares of initial, final, tone and typing differences; because three observed kinds have no operation, E is the weaker check. Each repeats the sweep from 10 to 50% on the same fragments.

Sensitivity to the way queries were made. The test-only variants of Section 3.4 change the prompt (Q4s), the information given to the writer (Q5g) and the typing of noisy fragments

Table 3: Query sets. Lengths in syllables. *Overlap*: share of the query's unmarked syllables that occur in the target song's lyrics, and in the lyrics of a random other song. *Run*: longest run of consecutive query syllables copied from the lyrics. *Unmarked*: share of queries typed without any diacritic. Statistics on the test split.

		queries				overlap			
Type	Description	train	dev	test	Length	target	random	Run	Unmarked
Q1	exact fragment	8542	603	1241	8.5	1.00	0.41	8.5	1%
Q2	unmarked fragment	8542	603	1241	8.5	0.97	0.40	8.0	100%
Q3	misheard / mistyped	8542	603	1241	8.5	0.80	0.34	4.8	1%
Q4	paraphrase (LLM)	–	75	227	17.2	0.70	0.41	2.2	0%
Q5	description (LLM)	–	75	227	27.2	0.72	0.35	3.4	0%
Q4u	Q4 typed without marks	–	75	227	17.2	0.70	0.41	2.2	100%
Q5u	Q5 typed without marks	–	75	227	27.2	0.72	0.35	3.4	100%
Q3u	Q3 typed without marks	–	0	1241	8.5	0.78	0.34	4.6	100%
Q4s	short paraphrase, other prompt	–	0	199	10.7	0.70	0.39	1.9	0%
Q5g	description from a gist	–	0	224	25.0	0.59	0.38	2.1	0%

(Q3u). For each we compare the effectiveness of the systems with that on the type it varies, on the same songs, and the orderings of the systems.

Agreement between orderings and discriminative power. Following Voorhees [48], we compute Kendall's τ between the orderings of the systems by MRR@10 on each query type and on the macro average of the other main types, between the macros of the rule-based (Q1–Q3) and the LLM-written types (Q4, Q5), between clean fragments and each noise level, and between channels at equal noise rates. Orderings with $\tau \geq 0.9$ are usually considered equivalent; we call values from 0.5 to 0.9 moderate agreement. We also replace the equal weights of the macro average by weights drawn from a flat Dirichlet distribution (1000 draws) and measure how the ordering moves. Discriminative power, the share of system pairs that a two-sided paired randomization test on reciprocal rank separates at $\alpha = 0.05$ [44], measures how sensitive a query set is, not whether its conclusions are right. Systems with several seeds count once (seed 1), runs over a sample of the queries are left out, and runs whose rankings are identical on every test query count once (Appendix B).

Clusters outside the pilot catalogue. The method was developed on the pilot catalogue, whose clusters also hold test queries. Section 7 repeats the main comparisons on the test queries about the other clusters.

4 Retrieval methods

4.1 Text analysis and lexical models

Text is normalised to NFC, lowercased and stripped of punctuation. Nine analyzers turn it into index terms: syllables; syllables plus adjacent syllable bigrams (*uni+bigrams*); words from the pyvi segmenter [47]; unmarked syllables; unmarked uni+bigrams; character 3-grams inside unmarked syllables, padded with boundary symbols; character 4-grams and 5-grams over the whole unmarked text, spaces included, so that a gram spans the boundary between two syllables (*yeu_e*); and a *sound-alike key* with bigrams, in the spirit of phonetic matching for misheard lyrics [51], which maps each syllable to a key that merges the onsets *x/s*, *ch/tr*, *n/l* and *d/gi/r/v*, the spellings *k/q/c* and *gh/g*, the codas *ng/n*, *nh/n* and *c/ch/t*, the tones *ngã* and *hỏi*, and *y* with *i*. The key merges the confusions that the hand-set noise channel injects, so its results under that channel are optimistic; the observed channel (Section 3.7) tests it fairly. Bigrams

keep local word order without segmentation. We use BM25 [42] with idf $= \log(1 + (N - n_t + 0.5)/(n_t + 0.5))$ and tune $k_1 \in \{0.1, 0.2, 0.3, 0.6, 0.9, 1.2, 1.5, 2.0\}$ and $b \in \{0.3, 0.5, 0.75, 0.9, 1.0\}$ per analyzer on the development queries; query likelihood with Dirichlet smoothing [52], $\mu \in \{5, 10, 20, 30, 50, 100, 300, 1000, 2000\}$ (tuned: $\mu = 5$); and RM3 pseudo-relevance feedback [1, 28], which takes the feedback songs from the query-likelihood ranking and scores the expanded query with BM25, with 5, 10 or 20 feedback songs, 10, 20 or 40 expansion terms and an original-query weight of 0.5, 0.7 or 0.9 tuned on development queries (tuned: 10 songs, 40 terms, weight 0.5), on syllables, unmarked syllables and syllable uni+bigrams.

A second field holds the title, artist and genre. We report BM25 on this field alone and a weighted sum of the field scores, $\text{BM25}_{\text{lyrics}} + w \cdot \text{BM25}_{\text{meta}}$ with $w \in \{0.02, 0.05, 0.1, 0.3, 0.5, 1\}$ tuned on development queries ($w = 0.1$), both on unmarked uni+bigrams. This simple combination is not BM25F [43], which we leave for future work.

4.2 Diacritic restoration

Restoration decodes an unmarked query as a noisy channel. Each unmarked syllable can come from any marked syllable that strips to it, up to the [?] most frequent, which cover [?]% of the syllable tokens of the lyrics ([?] of the [?] unmarked syllables have more forms), and an interpolated Kneser–Ney trigram language model [15, 27] with absolute discount 0.75, trained on lyric lines, chooses the most probable sequence with a beam of eight. Syllables that already carry marks are kept. A syllable that is not in the unmarked vocabulary is tried as a Telex key sequence (*aa*→â, *aw*→ă, *ow*→ơ, *uw*→ư, *dd*→đ, a final *s, f, r, x* or *j* gives the tone); its candidates are the vocabulary syllables with that toneless form and tone. We train the language model in two ways. *Restore*$_{\text{all}}$ uses all indexed lyrics, as a deployed engine would, since it holds its catalogue anyway. *Restore*$_{\text{train}}$ uses training songs only, so that the test songs are new to the model; it is a lower bound for songs that enter the catalogue after the model was trained. The restored query is searched with BM25 on marked syllables or uni+bigrams.

4.3 Query-type classification

A classifier predicts the type of a query (Q1–Q5, Q4u, Q5u) so that fusion can use type-specific weights. Its features are TF-IDF weights of character 1–4-grams within word boundaries (sublinear term frequency) and four scalar features: the share of syllables with marks, the shares of syllables outside the marked and the unmarked index vocabularies, and the length. We compare logistic regression ($C = 4$) and multinomial Naive Bayes ($\alpha = 0.1$). Training examples are the Q1–Q3 queries of training songs plus the development Q4, Q5, Q4u and Q5u queries, since these types exist only for development and test songs. Each rule type therefore has 8542 examples and each other type 75, about 114 times fewer. Fusion weights are tuned on development queries by their predicted type, so the predictions for development queries must not come from a model that saw them: each development query is classified by a model trained without the development queries of its own fold (five folds of clusters; accuracy 89.7%), and every other query by the model trained on all examples. We weight each class by the inverse of its frequency (for Naive Bayes, a uniform prior) when this wins 5-fold cross-validation on the training queries, with folds grouped by song; it does for both models (macro-F1 0.781 against 0.727 for logistic regression, 0.401 against 0.399 for Naive Bayes). An ablation removes the length feature (Section 6.3).

4.4 Dense retrieval

We evaluate three bi-encoders without fine-tuning: multilingual E5 small and base [50] and the PhoBERT-based bkai bi-encoder [10], which expects word-segmented input; its queries and passages are segmented with pyvi. We could not evaluate larger encoders such as BGE-M3 [14] and the Vietnamese_Embedding model of AITeamVN [2] (568M parameters each): encoding the

catalogue with them was beyond our CPU budget, and the model repository was not reachable from our computing environment when we revised the study. The released scripts run them on a GPU. Passages (Section 3.1) are encoded with at most 128 tokens, using the query and passage prefixes each model expects; a song's score is the highest cosine similarity of its passages (MaxP, [17]), and search is exact. Each model is run twice: on the queries as typed, and on the queries restored by $\text{Restore}_{\text{all}}$ (Section 4.2). The second run is the pipeline alternative to NA-FT: it asks whether marks restored before encoding help a bi-encoder as they help BM25. The form with the higher development score enters the hybrid runs.

4.5 Noise-aware fine-tuning

NA-FT adapts a bi-encoder to lyrics and noisy queries using training songs only and no LLM. It pairs (i) every Q1, Q2 and Q3 query of a training song with the passage that contains its fragment, and (ii) noisy inverse cloze examples [29]: one line of a passage, left unchanged, stripped of marks (30%) or corrupted at 10–30% (20%), paired with the other lines of the passage (six per song). Each pair gets one hard negative, drawn from the ten highest-ranked passages of other clusters under BM25 on unmarked uni+bigrams; positives and negatives are passages of training songs only. The loss is the multiple negatives ranking loss [20] with in-batch negatives; we train one epoch with batch size 32, learning rate $3 \cdot 10^{-5}$, 10% warm-up and three seeds. Two ablations, trained with seed 1 only and compared with NA-FT seed 1, remove the noise (the clean fragment replaces each rule query and inverse cloze queries stay clean) and the hard negatives (a random passage of another cluster). NA-FT models are also run on restored queries. NA-FT sees no paraphrase or description: it uses no LLM, and its only sentence-level pairs are the inverse cloze examples.

We fine-tune multilingual E5 small, the model that fits our CPU budget. To let training run while the crawl was still collecting songs, the pairs come from a frozen snapshot of the collection ([?] training songs), and each model is trained on a random subset of 10000 pairs (51% inverse cloze); one epoch takes about 3 minutes on four CPU cores. A song's split depends only on its cluster's key, but clusters can merge as the catalogue grows: [?] of the snapshot's training songs belong to development or test clusters of the final collection, and the [?] queries about these clusters are removed from the evaluation of every system.

4.6 Rank fusion with query-type-adaptive weights

RRF [16] scores a song d by $\sum_r w_r/(60 + \text{rank}_r(d))$ over the top 100 of each list r. The lexical fusion uses four lists: $\text{Restore}_{\text{all}}$ with uni+bigrams, unmarked uni+bigrams, character 3-grams and the metadata field. We compare equal weights; one weight vector tuned over the grid $\{0, 0.5, 1, 2\}^4$ on all development queries pooled, Q4u and Q5u included (ties resolved towards uniform weights); *adaptive* weights, one vector per type predicted by the classifier; and *oracle* weights per true type. Type-specific weights are tuned on few queries and may overfit, so a type keeps its own weights only if they beat the global vector in five-fold cross-validation over the development queries of that type. A hybrid adds a dense list to the four lexical lists. Its lexical weights stay global and only the weight of the dense list is adapted per type, over $w \in \{0, 0.05, 0.1, 0.25, 0.5, 1, 2, 4, 8, 16\}$. Among the hybrids, the one with the best development macro average over the seven typed query types is selected for every later comparison.

Other ways to add the dense list. We compare type-adaptive RRF with four alternatives, each tuned on development queries: (i) RRF of the tuned lexical fusion and the dense list with weights $(1, w)$, one w for all queries or one per predicted type under the same cross-validation guard; (ii) a convex combination $(1 - a)\,\hat{s}_{\text{lex}} + a\,\hat{s}_{\text{dense}}$ of the min–max normalised scores of the lexical fusion and the dense list [13], $a \in \{0, 0.05, \ldots, 1\}$, likewise global or per type; (iii) routing, which sends each predicted type to whichever of the lexical fusion, the dense list and the global

hybrid is best on the development queries of that type; and (iv) a selector that needs no query type, in the spirit of Arabzadeh et al. [4]: the dense list joins with weight w only when the relative margin $(s_1 - s_2)/s_1$ between the scores of the first two songs of the restored bigram run is below θ, with θ among the deciles of the development margins.

4.7 Cross-encoder reranking

The 20 best songs of the selected hybrid are reranked by a cross-encoder, bge-reranker-v2-m3 [14]. For each song we score its two passages closest to the query under the dense list of the hybrid; if that list uses restored queries, the cross-encoder sees the restored query too. Songs below rank 20 keep their order. On our CPU the cross-encoder scores about 275.4 pairs per second, so we rerank a stratified random sample of [?] test and [?] development queries of each of the seven typed types and compare with the first stage on the same queries. Besides the cross-encoder order, we tune on the development sample an interpolation $\alpha\,\hat{s}_{\mathrm{CE}} + (1-\alpha)\,\hat{s}_{\mathrm{first}}$ of min–max normalised scores and a selective variant that reranks only the predicted types for which reranking helped on the development sample.

5 Experimental setup

The main metric is MRR@10 of the target cluster, with success at 1 and 10 (S@1, S@10) and recall at 100 (R@100, the reach of a first-stage ranker). Because the rule-based types have more queries than the LLM types, we average each metric over the five synthetic types (*macro*) rather than over queries. We test differences with a two-sided paired randomization (sign-flip) test with 10,000 permutations [46], corrected with Holm's method [22] over the systems compared in a table column, or over the planned comparisons of Table 10. Intervals for a macro average are 95% bootstrap intervals with queries resampled within each type (2,000 draws); for a single type, the half-width of a 95% interval of MRR@10 is at most 0.059 for the selected hybrid. All parameters, fusion weights and model choices are selected on the development split. All experiments ran on a cloud virtual machine with four CPU cores and 15 GB of memory, without a GPU; passage vectors are cached by model weights, so each passage is encoded once per model. The lexical index is a sparse document–term matrix whose columns act as posting lists, and dense search is exact.

6 Results

All results are on the test split of the full collection (6337 songs in 6213 clusters; 4177 queries of the five main types). Exact and unmarked fragments are close to the ceiling for the best systems, so most differences between them lie in noisy fragments and in the two LLM-written types. Table 4 reports the lexical systems, Table 8 the dense, fine-tuned, hybrid and reranked ones, and Table 10 the planned comparisons between them, each tested with Holm's correction.

6.1 Lexical retrieval

Table 4 compares the lexical systems.

Bigrams matter more than segmentation. Adding syllable bigrams raises MRR@10 on exact fragments from 0.840 to 0.999 and on noisy fragments from 0.581 to 0.935. Word segmentation with pyvi does not help (0.870 on Q1) and hurts on paraphrases (0.246 against 0.325 for syllables): a segmenter trained on prose splits lyrics inconsistently, and a bigram index captures local order without committing to a segmentation.

Table 4: Lexical retrieval on the test split (MRR@10).

	MRR@10 by query type						macro average		
System	Q1	Q2	Q3	Q4	Q5	Macro	S@1	S@10	R@100
queries	1241	1241	1241	227	227				
Diacritic-sensitive									
BM25, syllables (ref.)	0.840	0.057	0.581	0.325	0.802	0.521	0.467	0.642	0.776
BM25, syllable uni+bigrams	0.999†	0.135†	0.935†	0.344	**0.934**†	0.670	0.643	0.728	0.818
BM25, words (pyvi)	0.870†	0.048‡	0.552‡	0.246‡	0.813	0.506	0.459	0.606	0.741
QL Dirichlet, syllables	0.857†	0.071†	0.694†	**0.354**	0.838	0.563	0.512	0.669	0.797
BM25+RM3, syllables	0.841	0.064†	0.656†	0.334	0.826	0.544	0.513	0.617	0.741
BM25+RM3, syllable uni+bigrams	0.999†	0.143†	0.943†	0.330	0.919†	0.667	0.643	0.716	0.791
Diacritic-insensitive									
BM25, unmarked syllables	0.715‡	0.696†	0.423‡	0.242‡	0.680‡	0.551	0.488	0.690	0.876
BM25, unmarked uni+bigrams	**1.000**†	0.992†	0.929†	0.288	0.898†	0.821	0.795	0.872	0.936
BM25+RM3, unmarked	0.749‡	0.732†	0.503‡	0.229‡	0.680‡	0.579	0.535	0.673	0.837
BM25, char. 3-grams in syllables	0.616‡	0.566†	0.270‡	0.157‡	0.443‡	0.410	0.341	0.564	0.793
BM25, char. 4-grams across syllables	0.955†	0.944†	0.811†	0.196‡	0.677‡	0.717	0.674	0.805	0.892
BM25, char. 5-grams across syllables	0.989†	0.985†	0.941†	0.248‡	0.840	0.801	0.772	0.860	0.922
Sound-alike key									
BM25, sound-alike key uni+bigrams	0.999†	0.112†	**0.968**†	0.336	0.929†	0.669	0.642	0.727	0.811
Diacritic restoration									
$\text{Restore}_{\text{train}}$ + uni+bigrams	0.995†	0.975†	0.925†	0.336	0.929†	0.832	0.806	0.883	0.943
$\text{Restore}_{\text{all}}$ + syllables	0.838	0.817†	0.586	0.323	0.791	0.671	0.607	0.808	0.927
$\text{Restore}_{\text{all}}$ + uni+bigrams	0.999†	**0.996**†	0.933†	0.342	0.932†	0.840	0.818	0.886	0.945
Metadata									
BM25, title/artist/genre only	0.081‡	0.080†	0.067‡	0.023‡	0.388‡	0.128	0.103	0.185	0.280
BM25, lyrics + metadata	0.999†	0.991†	0.929†	0.296	0.924†	0.828	0.804	0.873	0.939
Lexical fusion (RRF)									
RRF, equal weights	0.746‡	0.742†	0.646†	0.274	0.830	0.648	0.525	0.857	0.935
RRF, weights tuned on dev	0.999†	0.995†	0.943†	0.344	0.930†	**0.842**	0.820	0.886	0.945
RRF, weights per predicted type	0.999†	0.995†	0.943†	0.336	0.930†	0.840	0.817	0.886	0.945
RRF, weights per true type (oracle)	0.999†	0.995†	0.942†	0.342	0.930†	0.841	0.819	0.887	0.945

†/‡: significantly better/worse than BM25 on syllables (paired randomization test, 10,000 permutations, Holm correction over the systems of the table within each column, $p < 0.05$). Bold: best value per column, oracle rows excluded. Macro: unweighted mean over the five query types. $\text{Restore}_{\text{all}}$ trains the diacritic restoration language model on every indexed song, $\text{Restore}_{\text{train}}$ on training songs only.

Missing marks are the dominant error. Systems that index marked text collapse on unmarked queries: BM25 on syllables reaches 0.057 on Q2 and 0.135 with bigrams. Removing marks from the index solves the problem (0.992 with unmarked uni+bigrams) at the cost of more ambiguous unigrams, which hurts unigram retrieval on exact fragments (0.715 against 0.840) but not bigram retrieval (1.000).

Restoration works, even for unseen songs. The restoration model recovers 98.3% of the syllables of the test Q2 queries when trained on all lyrics and 92.0% when the test songs are unseen, in under a millisecond per query. Context does most of the work: taking each syllable's most frequent marked form recovers 77.3%, a bigram model 95.6%. On unseen songs the trigram is no better than the bigram (92.0% against 91.7%), so its gain with $\text{Restore}_{\text{all}}$ comes from lyrics it has already seen. With uni+bigrams, restored queries are as effective as unmarked indexing (0.996 and 0.975 against 0.992), and the best single lexical system on the development split is $\text{Restore}_{\text{all}}$ + uni+bigrams (test macro MRR@10 0.840). Restoration keeps the marked index, which is better on paraphrases (0.342 against 0.288), so it is the more balanced choice.

Table 5: Paraphrases (Q4) and descriptions (Q5) as written, with marks, and typed without marks (Q4u, Q5u): the same queries with every mark removed. MRR@10 on the test split. ‡/†: the unmarked version is significantly worse/better for that system (paired randomization test, Holm correction within each column, $p < 0.05$).

System	Q4	Q4u	Q5	Q5u
BM25, syllables	0.325	0.012‡	0.802	0.062‡
BM25, syllable uni+bigrams	0.344	0.027‡	0.934	0.097‡
QL Dirichlet, syllables	0.354	0.018‡	0.838	0.063‡
BM25, unmarked uni+bigrams	0.288	0.288	0.898	0.898
$\text{Restore}_{\text{train}}$ + uni+bigrams	0.336	0.280‡	0.929	0.888‡
$\text{Restore}_{\text{all}}$ + uni+bigrams	0.342	0.294‡	0.932	0.911
BM25, lyrics + metadata	0.296	0.296	0.924	0.924
RRF, weights per predicted type	0.336	0.299‡	0.930	0.910

Character n-grams are the weakest analyzer. Character 3-grams, a standard remedy for typos, are the worst lyrics analyzer on every type (macro 0.410), including noisy queries (0.270 on Q3). Vietnamese syllables are short: the median unmarked syllable in the lyrics has 3 letters and 97% have two to five, so a padded trigram is shared by 64 distinct syllables on average, and a single changed letter destroys most of the trigrams of a short syllable. Syllable bigrams are a far more selective unit.

Smoothing and feedback help only noisy queries. RM3 improves BM25 on noisy fragments (0.656 against 0.581) but not on the other types. Query likelihood with Dirichlet smoothing is also better than BM25 on noisy fragments (0.694), and it is the best lexical model on paraphrases (0.354), although that difference is not significant.

Paraphrases defeat lexical matching. No lexical system exceeds 0.354 on Q4. Paraphrases share words but few exact bigrams with the lyrics, and without marks shared words become ambiguous: unmarked unigrams reach only 0.242. These queries are where a semantic model is needed.

Descriptions benefit from metadata. The metadata field alone retrieves the right song for many descriptions (0.388 on Q5) and almost nothing else; added to the lyrics it gives the best result on Q5 (0.924), because descriptions often name the singer or the composer.

Paraphrases lose more than descriptions without marks. Table 5 repeats Q4 and Q5 with every mark removed. Systems that index marked text lose almost everything, as on Q2: BM25 on syllables falls from 0.325 to 0.012 on paraphrases and from 0.802 to 0.062 on descriptions. Unmarked indexing is unaffected by construction. Restoration recovers most of the loss on descriptions (0.911 against 0.932, not significantly lower) but less on paraphrases (0.294 against 0.342). It restores 93.9% of the syllables of Q4u queries and 92.2% of those of Q5u, against 98.3% for Q2: an unmarked fragment is a sequence that the language model has seen in the lyrics, a paraphrase or a description is not. The best lexical system on Q4u is RRF, weights per predicted type (0.299). Dense retrieval behaves like the marked lexical indexes: mE5-base falls from [?] to [?] on unmarked paraphrases, and restoration brings it back to [?].

Fusion must be weighted. RRF with equal weights is much worse than its best component (macro 0.648): in known-item search a single song must be ranked first, and lists that are weak for a query type (the metadata list for lyric fragments, character trigrams everywhere) push it down. The weights tuned on all development queries, (2, 0.5, 0, 0) for the restored bigram, unmarked

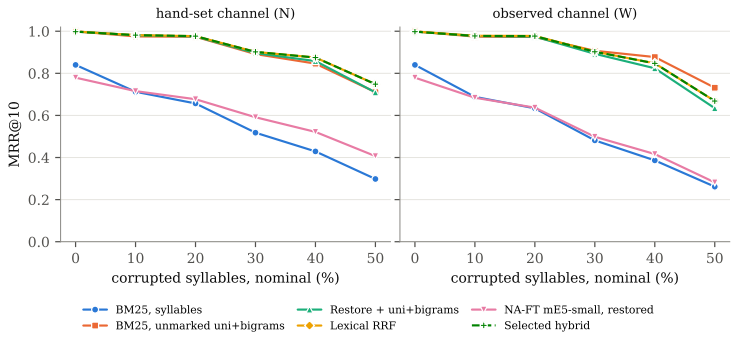


Figure 1: MRR@10 on the noise sweep: the same test fragments with 0–50% of their syllables corrupted by the noise channel.

bigram, character trigram and metadata lists, keep only the two bigram lists, and tuned fusion is no better than restoration alone (macro 0.842 against 0.840). The cross-validation guard kept type-specific weights for noisy fragments, (0.5, 2, 0, 0), which favour the unmarked list, and for paraphrases, (1, 0, 0, 0), both by small margins (cross-validated MRR@10 0.950 against 0.938, and 0.356 against 0.340). They do not carry over to the test split: even with the true query type, fusion reaches 0.942 on Q3 and 0.342 on Q4, against 0.943 and 0.344 with global weights. The lexical lists are too redundant for type-specific weights to matter; the dense list, which behaves differently from all of them, is where adaptivity pays off (Section 6.5).

6.2 Robustness to noise

Figure 1 and Table 6 trace MRR@10 as the share of corrupted syllables grows. BM25 on syllables falls from 0.840 to 0.298 at 50%, character trigrams from 0.616 to 0.118, while unmarked uni+bigrams keep 0.711 and restoration with uni+bigrams 0.709. Removing marks makes the index blind to tone errors, and with bigrams a corrupted syllable breaks only the two bigrams around it, so the intact stretches of the query still match. Dense models start lower and lose about as much: mE5-base on restored queries falls from [?] to [?], NA-FT from 0.779 to 0.407. The noise in the training data of NA-FT is what makes it more robust: trained on clean pairs, the same model (seed 1, queries as typed) keeps [?] at 50% noise instead of [?] (Table 10). The hybrid keeps the lexical result (0.749 at 50%), because noisy fragments are rarely taken for paraphrases, the only type that gives the dense list a weight (Section 6.5). Under the channel estimated from real transcriptions every system loses about as much (last column of Table 6; Section 6.6).

6.3 Query-type classification

Logistic regression predicts the query type with 89.4% accuracy (macro-F1 0.858), Naive Bayes with 74.6% (macro-F1 0.378; Table 7). Unmarked fragments and descriptions are almost always recognised (recall 99% and 98%). Noisy fragments are confused with exact ones (recall 81%) when the noise produces valid syllables, for instance *sương→xương*. Paraphrases remain hard (recall 55%): the classifier has only 75 training examples of them, and a paraphrase looks like a lyric line. Class weighting raises this recall from 19% and the accuracy from 86.7%. For the same reason unmarked paraphrases are often taken for unmarked fragments (recall 59%), while unmarked descriptions are recognised (99%). Confusing a paraphrase with a fragment matters because the hybrid gives the two types different weights (Section 6.5), and it is the main cause of the paraphrases that the hybrid misses (Section 6.7).

Table 6: MRR@10 on the noise sweep: the same 1241 test fragments with a growing share of corrupted syllables, under the hand-set channel (N), the channel sampled from the variation observed between transcriptions (W) and the hand-set operations re-weighted by the observed shares (E, 50% only; Section 3.7). Columns give the nominal rate; the realised share of changed syllables is in the last row. NA-FT: mean over three seeds.

		hand-set channel (N)					observed channel (W)					E
System	0%	10%	20%	30%	40%	50%	10%	20%	30%	40%	50%	50%
BM25, syllables	0.840	0.712	0.657	0.518	0.429	0.298	0.688	0.634	0.481	0.386	0.261	0.315
BM25, unmarked uni+bigrams	1.000	0.975	0.974	0.891	0.846	0.711	0.974	0.973	0.907	0.877	0.732	0.728
BM25, char. 3-grams in syllables	0.616	0.396	0.307	0.218	0.151	0.118	0.459	0.394	0.283	0.218	0.155	0.113
BM25, char. 5-grams across syllables	0.989	0.965	0.968	0.916	0.907	0.844	0.942	0.930	0.856	0.829	0.673	0.857
BM25, sound-alike key	0.999	0.988	0.987	0.943	0.931	0.867	0.979	0.974	0.887	0.832	0.633	0.825
Restore + uni+bigrams	0.999	0.981	0.975	0.896	0.858	0.709	0.977	0.974	0.893	0.824	0.634	0.713
Lexical RRF	0.999	0.981	0.977	0.902	0.875	0.749	0.978	0.978	0.903	0.847	0.668	0.748
NA-FT mE5-small, restored	0.779	0.715	0.677	0.591	0.522	0.407	0.684	0.637	0.499	0.416	0.282	0.414
Selected hybrid	0.998	0.981	0.977	0.902	0.875	0.749	0.978	0.977	0.902	0.847	0.668	0.748
realised (%)	0	13	19	31	40	50	13	19	31	40	50	50

Table 7: Confusion matrix of the query-type classifier (logistic regression) on the test queries. Rows: true type; columns: predicted type. Accuracy 89.4% (Naive Bayes: 74.6%). Q4u and Q5u are Q4 and Q5 typed without marks.

	predicted							
true	Q1	Q2	Q3	Q4	Q5	Q4u	Q5u	recall
Q1	1208	10	14	9	0	0	0	97%
Q2	7	1224	0	0	0	10	0	99%
Q3	224	9	1006	2	0	0	0	81%
Q4	83	0	1	124	19	0	0	55%
Q5	0	0	0	4	223	0	0	98%
Q4u	0	70	0	0	0	133	24	59%
Q5u	0	0	0	0	0	3	224	99%

6.4 Dense retrieval and noise-aware fine-tuning

Zero-shot bi-encoders handle paraphrases, not missing marks. Without fine-tuning, mE5-base is the best dense model (macro [?] on restored queries, Table 8). It is the best of all systems on paraphrases ([?], against 0.354 for the best lexical model), but far below lexical retrieval on fragments ([?] on Q1) and on descriptions ([?], against 0.924 for BM25 with metadata): a description is about the whole song, and a passage of four lines rarely contains what it describes. The bkai bi-encoder, trained on segmented prose, is the weakest model (macro [?]). All three collapse on unmarked fragments (mE5-base: [?] on Q2): for these encoders an unmarked query is a different string from its marked passage. Restoring the marks before encoding repairs this ([?] for mE5-base) and changes no other type (Table 10, first row), so every hybrid uses restored queries.

Noise-aware fine-tuning helps where lexical retrieval is already strong. NA-FT raises mE5-small from [?] to 0.667 macro MRR@10 on restored queries (mean of three seeds; per-seed macro from 0.661 to 0.670, standard deviation 0.005). The gain is significant on every fragment type and largest on noisy fragments ([?]) and at 50% noise ([?]); paraphrases gain [?], and descriptions lose [?] (not significant), as no training pair resembles a description. The fine-tuned small model is still worse than zero-shot mE5-base on paraphrases (0.540 against [?]) and descriptions. The ablations credit the noise with the gains: trained on clean pairs, seed 1

loses [?] on noisy fragments and [?] on unmarked ones, and gains [?] on descriptions, whereas random negatives in place of BM25-mined hard negatives change no type significantly (at most [?], on paraphrases). Training on unmarked queries is no substitute for restoration: NA-FT saw unmarked fragments in training but reaches only 0.121 on Q2 as typed, against 0.768 on restored queries.

6.5 Hybrid retrieval and reranking

A dense list helps only with weights per query type. Table 9 adds the dense list of the selected hybrid (NA-FT mE5-small, seed 3, the best seed on development queries) to the tuned lexical fusion with one weight w for every query. Every weight that helps paraphrases costs more on the other types: at $w = 8$ paraphrases reach [?], but noisy fragments fall to [?] and descriptions to [?]. The single weight with the best development macro, $w =$ [?], gains on paraphrases ([?] against [?]) but loses more on descriptions ([?] against [?]) and fragments, and the test macro falls from [?] to [?]; tuned on all development queries pooled, as in our global fusion, the weight is 0, and the hybrid with global weights is the lexical fusion (Table 8). With weights per type, the cross-validation guard gives the dense list a weight of [?] for queries predicted as [?] and none elsewhere. Paraphrases then rise from [?] to [?] while no other type changes (Table 10), and macro MRR@10 rises from 0.842 to 0.879. The classifier limits the gain: with the true query type, paraphrases reach [?] ([?]). A weight of 8, the largest of the grid, lets the dense model decide the ranking of a predicted paraphrase: any song in its top 100 outscores a song that both bigram lists rank first but the dense list misses ($8/160 > 2.5/61$), and the lexical lists only break ties.

Which dense model. On the development split the hybrids with NA-FT mE5-small and with zero-shot mE5-base are tied (macro 0.853 and [?]); on the test split mE5-base is better on paraphrases ([?] against [?]), as it is on its own, and the two are equal elsewhere. Fine-tuning on fragments improved the dense model where the hybrid does not need it.

Reranking. The cross-encoder reranks the top 20 songs of the selected hybrid for a stratified sample of 2531 test queries (40 of each type, unmarked types included). On paraphrases it adds [?] ([?] to [?], significant even on 40 queries), but it loses on every other type, from [?] on unmarked fragments to [?] on descriptions (none significant on 40 queries), and the macro average falls from 0.880 to 0.848. Scoring a single passage per song, the cross-encoder judges meaning, which suits paraphrases, but it ignores the exact word sequence that the first stage matches in a fragment and the rest of the song that a description is about. Reranking would therefore pay off only for queries predicted as paraphrases, and at 0.1 s per query on our CPU it is by far the most expensive component (Section 6.9).

6.6 Validity of the synthetic queries

The 116 pairs of versions align 32,019 syllables, of which 349 are substituted (10.9 per thousand); 47% of the substitutions are different words, which only Q4 imitates. Among the 185 substitutions of the kinds the channel models (Table 11), tone changes (34%) and typing differences (40%) are more frequent than the hand-set weights assume, and final consonants much rarer (2% against 15%). Only 20% of the initial changes are the dialectal mergers of the channel; the others keep the rhyme and change the word (*muộn/buồn*, *nồng/mộng*), as misheard lyrics do. The real transcriptions thus contain more tone and typing differences and fewer coda changes than the channel assumes, but repeating the sweep with the estimated weights changes little (last column of Table 6): over the 43 systems and the five noise levels, MRR@10 changes by +0.005 on average and by at most 0.042, and the orderings of the systems at equal rates agree

closely (Kendall's τ 0.87 on average, at least 0.81). The conclusions about noise do not depend on the hand-set weights.

Agreement between query types. The rule-based types order the 43 systems much like the macro average of the other types (Table 12; τ from 0.40 for exact fragments to 0.57 for noisy ones), and so do descriptions (0.46). The ordering on clean fragments survives heavy noise (τ 0.55 between 0% and 50%). Paraphrases are the exception: their ordering is unrelated to that of the other types (τ −0.02), and the lowest agreement of any two types is between exact fragments and paraphrases (τ −0.06). This follows from the results rather than from a flaw of the generator: dense models are the worst systems on fragments and the best on paraphrases, so a collection would rank systems very differently depending on which kind of query it holds. The rule-based and LLM-written families agree moderately (τ 0.25). A collection that leaves out paraphrases, or has only paraphrases, would reach different conclusions about dense retrieval.

Discriminative power. Every type separates most of the 903 pairs of systems at $\alpha = 0.05$ (78–90% for the main types, 80% for unmarked paraphrases). The 227 paraphrases and descriptions separate systems almost as often as the 1241 fragments of each rule-based type, partly because many systems tie at the ceiling on fragments.

6.7 Error analysis

We examined the test queries that the selected hybrid misses (target below rank 10). It misses [?] exact and [?] unmarked fragments.

Noisy fragments fail [?] times out of [?]. The failures are short ([?] syllables against [?] for the others), and the noise leaves them few syllables of the lyrics ([?] of their unmarked syllables occur in the target, against [?]) and short intact runs ([?] against [?] syllables). In *bao gnày nghe tiến mưa*, from *bao ngày nghe tiếng mưa* ('so many days hearing the rain'), a transposition and a coda merger leave three common syllables, which occur together in many songs.

Paraphrases fail [?] times out of [?]. The classifier took [?] of these for exact fragments and [?] for descriptions, so the dense list got no weight; only [?] were recognised and still missed. Missed paraphrases are shorter ([?] against [?] syllables) and share fewer syllables with the lyrics ([?] against [?]): the more freely a paraphrase is worded, the more it reads like the common language of love songs. Medleys, which hold the lines of several songs, rank first for [?]% of the missed paraphrases and [?]% of the others, so they are not a main source of error.

Descriptions fail [?] times. They quote the lyrics less than the descriptions that succeed (overlap [?] against [?]; longest copied run [?] against [?] syllables) and describe the song in general terms. [?] of them name the singer or composer, but the tuned fusion gives the metadata list no weight, and *bài vui tươi của sơn tùng về lần đầu rung động…* ('a cheerful song by Sơn Tùng about falling in love for the first time…') ranks another song by the same singer above the target. A metadata weight for queries predicted as descriptions is an obvious refinement.

6.8 Effect of catalogue size

Our catalogue is small next to a commercial service. To see how the results depend on its size, we shrink it: the index keeps a random share of the other clusters (the target cluster always stays), and each ranking is restricted to the kept songs (Table 13). The restricted rankings keep the term statistics of the full index, a small advantage for BM25 in the smaller catalogues. Exact fragments are unaffected for bigram indexes (the hybrid reaches [?] on a quarter of the catalogue and [?] on all of it), whereas unigrams lose [?] per doubling of the catalogue. Paraphrases are

the most sensitive: each doubling costs the hybrid about [?] on Q4 ([?] on a quarter, [?] on the full catalogue), and mE5-base alone loses as much ([?] per doubling). Descriptions lose [?] per doubling. In a catalogue many times larger, fragments would thus remain largely solved while the gap to paraphrases would widen, and our results on paraphrases and descriptions are best read as upper bounds for larger catalogues.

6.9 Efficiency

Table 14 gives the costs on our machine. Lexical search takes well under a millisecond per query (0.4 ms for unmarked uni+bigrams) and an index is built in seconds; restoration adds 0.6 ms per query. A dense model encodes the 79328 passages once in 13 (NA-FT mE5-small) to 47 minutes (mE5-base), a query in 5 to 15 ms, and exact search over the passage vectors takes 5 to 9 ms. Training NA-FT takes about 3 minutes per model. The selected hybrid therefore answers a query in about 11 ms on four CPU cores without a GPU, whereas the cross-encoder needs 0.1 s to rerank 20 songs.

7 Discussion and limitations

What the results suggest for Vietnamese song search. Index syllable bigrams over unmarked text, or restore the marks of a query and search marked bigrams; do not rely on character n-grams for tolerance to typing errors; restore marks before any dense encoder; and give a dense model weight only for queries that look like paraphrases. The gain then depends on recognising paraphrases, which our classifier does for about half of them; a classifier trained on more varied paraphrases, or probabilities instead of a hard decision, are the natural next steps. NA-FT, which needs no LLM and trains in about 3 minutes on a CPU, is worth its cost for a dense-only engine, not for a hybrid.

Catalogue size and source. The catalogue has 6337 songs, [?]% of them a uniform sample of one chord site, so songs that nobody has transcribed for guitar are missing, and a commercial service holds far more songs. We stopped the crawl at 8234 songs and did not add distractor songs, so the effect of a larger catalogue is extrapolated from smaller ones: Section 6.8 shows that exact and unmarked fragments are robust to the size of the catalogue while paraphrases and descriptions are not.

Synthetic queries. Fragments are cut from the lyrics, which favours lexical models, and the ambiguity filter removes fragments shared by two songs, which may remove some of the lines people remember best. Paraphrases and descriptions written by an LLM may differ from people's in vocabulary and style. All of them come from one model family; the prompt and writer variants (Q4s, Q5g) show how far the conclusions depend on the prompt and on what the writer saw, but not what another generator would write. The Q5 writer saw the lyrics and the artist, which makes these descriptions more precise than those of a person who heard the song once; Q5g, written from a gist alone, is the more realistic description set. Q4u and Q5u only remove marks, while people who type without marks may also write shorter or differently worded queries. Queries written by an LLM can also favour retrieval models that resemble it [39]; none of our retrieval models is an LLM, but the bi-encoders were trained on similar web text. The checks of Section 3.7 show whether conclusions depend on the query type, the prompt or the channel, but not whether people search the same way; real queries, for instance from the logs of a music service or from a questionnaire, would show that.

Noise channel. The confusion classes of the hand-set channel come from dialectology, but their probabilities and the weights of the operations were set by hand. The observed channel

replaces them with the kinds of substitution found between transcriptions, but these come from people writing down lyrics they know, at about one error per hundred syllables; the error rates of people who recall a lyric, and errors such as abbreviations (*ko* for *không*) and mixed scripts, remain unmodelled. Lexical normalisation corpora of Vietnamese social media text [32] could calibrate the typing part of the channel against real informal writing.

Models we could not evaluate. Larger multilingual and Vietnamese encoders (BGE-M3, Vietnamese_Embedding), Vietnamese rerankers [18] and neural diacritic restorers [19] were not available in our computing environment, which had no GPU and, during the revision, no access to the model repository. Our Kneser–Ney restorer already restores 98.3% of the syllables of unmarked fragments, so a neural restorer matters mostly for unmarked paraphrases and descriptions. NA-FT has no paraphrase-like training pairs by design (it uses no LLM); training a bi-encoder on paraphrases is the most direct way to attack the weakest query type.

Development data. The method was designed on the pilot catalogue, whose clusters also hold test queries. Section 7 shows the results on the other clusters.

Copyright and release. Lyrics are copyrighted. We release the code that rebuilds the corpus from the crawled pages, the song identifiers, source URLs, crawl dates and a checksum of each song's normalised lyrics (so that a re-crawl can be checked against ours), the queries and the run files, but not the lyrics. Queries are short fragments or paraphrases. The crawler identified itself as a research crawler in its user agent, fetched one page per second and followed the site's `robots.txt`. [Authors: check the terms of the source sites and the journal's data policy]

subsectionResults outside the pilot catalogue These results come from the round-1 analyses that are being rerun; the numbers marked [?] are filled in by that run.

8 Conclusion

ViSongSearch is a test collection for finding Vietnamese songs from imperfect memories of their lyrics, with queries that range from exact fragments to paraphrases and descriptions, and with validity checks that need no participants. Three conclusions stand out. First, the index unit and the handling of diacritics matter more than the retrieval model: syllable bigrams over unmarked or restored text retrieve exact, unmarked and noisy fragments almost perfectly and degrade gracefully with noise, whereas character n-grams, segmented words and pseudo-relevance feedback bring little or harm. Second, paraphrases remain the open problem. Bi-encoders handle them better than lexical models, but only when the diacritics are restored before encoding, and noise-aware fine-tuning improves a bi-encoder where lexical retrieval is already strong rather than where it is weak. Third, a dense model improves a hybrid only through weights that depend on the predicted query type, and the recognition of paraphrases now limits the gain. Better recognition of paraphrases, use of the metadata for descriptions, larger catalogues and real queries are the next steps.

Statements

Data and code availability. The code that rebuilds the collection from the source pages, the song identifiers and source URLs, the queries, the run files and the evaluation scripts are available at [Authors: repository URL]. Lyrics are not redistributed (Section 7).

Ethics. The study involves no human participants. The crawled pages are public, and only song lyrics and metadata were kept; no user accounts or comments were collected.

Use of AI tools. The code, the experiments and a first draft of this paper were produced with the help of an AI assistant (Claude, Anthropic), and the Q4 and Q5 queries were written by the same model (Section 3.6). Every number in the paper is computed by the released scripts from the run files. [Authors: the authors checked and take responsibility for the content; add model version if the venue requires it]

References

[1] Nasreen Abdul-Jaleel, James Allan, W. Bruce Croft, Fernando Diaz, Leah Larkey, Xiaoyan Li, Mark D. Smucker, and Courtney Wade. UMass at TREC 2004: Novelty and HARD. In *Proceedings of the Thirteenth Text REtrieval Conference (TREC 2004)*, 2004.

[2] AITeamVN. Vietnamese_Embedding and Vietnamese_Reranker. Hugging Face model cards, `https://huggingface.co/AITeamVN`, 2025.

[3] Marwah Alaofi, Luke Gallagher, Mark Sanderson, Falk Scholer, and Paul Thomas. Can generative LLMs create query variants for test collections? An exploratory study. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

[4] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting efficiency/effectiveness trade-offs for dense vs. sparse retrieval strategy selection. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, pages 2862–2866, 2021.

[5] Jaime Arguello, Adam Ferguson, Emery Fine, Bhaskar Mitra, Hamed Zamani, and Fernando Diaz. Tip of the tongue known-item retrieval: A case study in movie identification. In *Proceedings of the 2021 Conference on Human Information Interaction and Retrieval (CHIIR)*, 2021.

[6] Jaime Arguello, Adam Ferguson, Emery Fine, Bhaskar Mitra, Hamed Zamani, and Fernando Diaz. Overview of the TREC 2023 tip-of-the-tongue track. In *The Thirty-Second Text REtrieval Conference (TREC 2023)*, 2023.

[7] Leif Azzopardi, Maarten de Rijke, and Krisztian Balog. Building simulated queries for known-item topics: An analysis using six European languages. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2007.

[8] Samarth Bhargav, Georgios Sidiropoulos, and Evangelos Kanoulas. 'it's on the tip of my tongue': A new dataset for known-item retrieval. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining (WSDM)*, 2022. doi: 10.1145/3488560.3498421.

[9] Samarth Bhargav, Anne Schuth, and Claudia Hauff. When the music stops: Tip-of-the-tongue retrieval for music. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

[10] BKAI Foundation Models. vietnamese-bi-encoder. Hugging Face model card, `https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder`, 2023.

[11] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. InPars: Unsupervised dataset generation for information retrieval. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022. doi: 10.1145/3477495.3531863.

[12] Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of SEQUENCES 1997*, pages 21–29, 1997.

[13] Sebastian Bruch, Siyu Gai, and Amir Ingber. An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, 42(1), 2023.

[14] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[15] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394, 1999.

[16] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[17] Zhuyun Dai and Jamie Callan. Deeper text understanding for IR with contextual neural language modeling. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 985–988, 2019.

[18] Phuong-Nam Dang, Kieu-Linh Nguyen, and Thanh-Hieu Pham. ViRanker: A BGE-M3 & blockwise parallel transformer cross-encoder for Vietnamese reranking. arXiv:2509.09131, 2025.

[19] Trung Duc Anh Dang and Thi Thu Trang Nguyen. TDP – a hybrid diacritic restoration with transformer decoder. In *Proceedings of the 34th Pacific Asia Conference on Language, Information and Computation (PACLIC)*, pages 76–83, Hanoi, Vietnam, 2020.

[20] Matthew Henderson, Rami Al-Rfou, Brian Strope, Yun-Hsuan Sung, László Lukács, Ruiqi Guo, Sanjiv Kumar, Balint Miklos, and Ray Kurzweil. Efficient natural language response suggestion for smart reply. arXiv:1705.00652, 2017.

[21] Hussein Hirjee and Daniel G. Brown. Solving misheard lyric search queries using a probabilistic model of speech sounds. In *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, 2010.

[22] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[23] Thị Châu Hoàng. *Phương ngữ học tiếng Việt [Vietnamese Dialectology]*. NXB Đại học Quốc gia Hà Nội, Hanoi, 2004.

[24] In-Ho Kang and GilChang Kim. Query type classification for web document retrieval. In *Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 64–71, 2003.

[25] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[26] James Kirby. Vietnamese (Hanoi Vietnamese). *Journal of the International Phonetic Association*, 41(3):381–392, 2011. doi: 10.1017/S0025100311000181.

[27] Reinhard Kneser and Hermann Ney. Improved backing-off for m-gram language modeling. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[28] Victor Lavrenko and W. Bruce Croft. Relevance-based language models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 120–127, 2001.

[29] Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. Latent retrieval for weakly supervised open domain question answering. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 6086–6096, 2019.

[30] Dat Quoc Nguyen and Anh Tuan Nguyen. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1037–1042, 2020.

[31] Quoc Anh Nguyen, Bernard Cheng, and Kelvin Soh. VietLyrics: A large-scale dataset and models for Vietnamese automatic lyrics transcription. arXiv:2510.22295, 2025.

[32] Thanh-Nhi Nguyen, Thanh-Phong Le, et al. ViLexNorm: A lexical normalization corpus for Vietnamese social media text. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

[33] Vinh Nguyen, Nam Tran, Long Nguyen, and Dien Dinh. Advancing Vietnamese information retrieval with learning objective and benchmark. In *Proceedings of the 38th Pacific Asia Conference on Language, Information and Computation (PACLIC)*, pages 46–56, Tokyo, Japan, 2024.

[34] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019.

[35] Paul Ogilvie and Jamie Callan. Combining document representations for known-item search. In *Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 143–150, 2003.

[36] Gustavo Penha, Arthur Câmara, and Claudia Hauff. Evaluating the robustness of retrieval pipelines with query variation generators. In *Advances in Information Retrieval: 44th European Conference on IR Research (ECIR 2022)*. Springer, 2022.

[37] Loc Pham, Tung Luu, Thu Vo, Minh Nguyen, and Viet Hoang. VN-MTEB: Vietnamese massive text embedding benchmark. arXiv:2507.21500, 2025.

[38] Thai-Hoang Pham, Xuan-Khoai Pham, and Phuong Le-Hong. On the use of machine translation-based approaches for Vietnamese diacritic restoration. arXiv:1709.07104, 2017.

[39] Hossein A. Rahmani, Nick Craswell, Emine Yilmaz, Bhaskar Mitra, and Daniel Campos. Synthetic test collections for retrieval evaluation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[40] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[41] Nicholas Ring and Alexandra L. Uitdenbogerd. Finding 'Lucy in disguise': The misheard lyric matching problem. In *Information Retrieval Technology: 5th Asia Information Retrieval Symposium, AIRS 2009*, pages 157–167. Springer, 2009. doi: 10.1007/978-3-642-04769-5_14.

[42] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[43] Stephen Robertson, Hugo Zaragoza, and Michael Taylor. Simple BM25 extension to multiple weighted fields. In *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management (CIKM)*, pages 42–49, 2004.

[44] Tetsuya Sakai. Evaluating evaluation metrics based on the bootstrap. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 525–532, 2006. doi: 10.1145/1148170.1148261.

[45] Georgios Sidiropoulos and Evangelos Kanoulas. Analysing the robustness of dual encoders for dense retrieval against misspellings. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

[46] Mark D. Smucker, James Allan, and Ben Carterette. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the Sixteenth ACM Conference on Information and Knowledge Management (CIKM)*, pages 623–632, 2007.

[47] Viet Trung Tran. pyvi: Python Vietnamese core NLP toolkit. `https://github.com/trungtv/pyvi`, 2018.

[48] Ellen M. Voorhees. Variations in relevance judgments and the measurement of retrieval effectiveness. *Information Processing & Management*, 36(5):697–716, 2000.

[49] Thanh Vu, Dat Quoc Nguyen, Dai Quoc Nguyen, Mark Dras, and Mark Johnson. VnCoreNLP: A Vietnamese natural language processing toolkit. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, pages 56–60, 2018.

[50] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings: A technical report. arXiv:2402.05672, 2024.

[51] Xin Xu, Masaki Naito, Tsuneo Kato, and Hisashi Kawai. Robust and fast lyric search based on phonetic confusion matrix. In *Proceedings of the 10th International Society for Music Information Retrieval Conference (ISMIR)*, pages 417–422, 2009.

[52] Chengxiang Zhai and John Lafferty. A study of smoothing methods for language models applied to ad hoc information retrieval. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 334–342, 2001.

[53] Shengyao Zhuang and Guido Zuccon. Dealing with typos for BERT-based passage retrieval and ranking. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[54] Shengyao Zhuang and Guido Zuccon. CharacterBERT and self-teaching for improving the robustness of dense retrievers on queries with typos. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

A Instructions for LLM-written queries

Instructions were given in Vietnamese; English translations follow each.

Q4 (paraphrase). *Bạn là người nghe nhạc chỉ nhớ mang máng một đoạn lời bài hát tiếng Việt. Đây là đoạn lời thật: "{đoạn lời}". Hãy viết lại đoạn này như cách bạn nhớ sai: giữ ý chính, nhưng đổi phần lớn từ ngữ (dùng từ đồng nghĩa, đảo trật tự, bỏ bớt chữ). Không chép nguyên cụm từ 4 chữ trở lên. Viết 1 câu, 8–20 âm tiết, chữ thường, không dấu câu. Chỉ trả lời câu đó.*
You are a listener who only vaguely remembers a fragment of a Vietnamese song. Here is the real fragment: "{fragment}". Rewrite it the way you misremember it: keep the main idea but change most of the words (synonyms, different order, fewer words). Do not copy any phrase of four syllables or more. Write one sentence of 8–20 syllables, lowercase, without punctuation. Answer with that sentence only.

Q5 (description). *Bạn đang cố tìm một bài hát tiếng Việt nhưng quên tên bài. Thông tin bạn còn nhớ (KHÔNG được nhắc tên bài): Tên bài (chỉ để bạn biết, cấm viết ra): {tên}; Sáng tác: {nhạc sĩ}; Ca sĩ: {ca sĩ}; Lời bài hát (một phần): {lời}. Hãy viết một câu tìm kiếm như người thật gõ vào Google khi quên tên bài: mô tả nội dung, cảm xúc, hoàn cảnh, có thể nhớ lờ mờ vài chữ hoặc tên ca sĩ. Không viết tên bài. 1–2 câu, tối đa 35 âm tiết, chữ thường. Chỉ trả lời câu đó.*
You are trying to find a Vietnamese song but have forgotten its title. What you remember (do NOT mention the title): title (for your information only, never write it): {title}; composer: {composer}; singers: {singers}; part of the lyrics: {lyrics}. Write a search query as a real person would type into Google after forgetting the title: describe the content, feeling or situation; you may vaguely remember a few words or the singer. Do not write the title. One or two sentences, at most 35 syllables, lowercase. Answer with the query only.
For the pilot clusters, whose sources keep a single artist field, the composer and singer lines were one line, *Tác giả / ca sĩ: {nghệ sĩ}*. The filters of Section 3.6 are looser than the instruction not to copy four syllables: they reject a copied run of five syllables or more for Q4.

Q4s (short paraphrase, test only). *Bạn là người nghe nhạc chỉ nhớ mang máng một đoạn lời bài hát tiếng Việt. Đây là đoạn lời thật: "{đoạn lời}". Hãy viết lại đoạn này như cách bạn nhớ sai: giữ ý chính, nhưng đổi phần lớn từ ngữ (dùng từ đồng nghĩa, đảo trật tự, bỏ bớt chữ). Không chép nguyên cụm từ 4 chữ trở lên. Viết 1 câu ngắn, 5–12 âm tiết, chữ thường, có dấu tiếng Việt, không dấu câu. Chỉ trả lời câu đó.*
As for Q4, but: write one short sentence of 5–12 syllables, lowercase, with Vietnamese marks, without punctuation.

Gist (first step of Q5g). *Đọc lời bài hát dưới đây và tóm tắt nội dung của bài trong 1 câu, tối đa 20 âm tiết: bài nói về ai, chuyện gì, cảm xúc và hoàn cảnh ra sao. Viết bằng lời của bạn: không dùng lại cụm từ 3 chữ liên tiếp trở lên của lời bài hát, không trích lời. Chữ thường, có dấu tiếng Việt. {lời}*
Read the lyrics below and summarise what the song is about in one sentence of at most 20 syllables: who it is about, what happens, the feelings and the situation. Use your own words: do not reuse any run of three or more consecutive syllables of the lyrics, do not quote. Lowercase, with Vietnamese marks. {lyrics}

Q5g (description from the gist, test only). *Bạn đang cố tìm một bài hát tiếng Việt nhưng quên tên bài và không nhớ lời. Bạn chỉ nhớ bài hát nói về: "{tóm ý}". Hãy viết một câu tìm kiếm như người thật gõ vào Google khi quên tên bài: mô tả nội dung, cảm xúc, hoàn cảnh. 1–2 câu, tối đa 35 âm tiết, chữ thường, có dấu. Chỉ trả lời câu đó.*
You are trying to find a Vietnamese song but have forgotten its title and do not remember the lyrics. You only remember that the song is about: "{gist}". Write a search query as a real person

would type into Google after forgetting the title: describe the content, feeling or situation. One or two sentences, at most 35 syllables, lowercase, with marks. Answer with the query only.

B Systems compared in the validity analyses

The agreement and discriminative-power analyses (Table 12) use the following 43 runs over every test query; 4 further runs whose rankings are identical to one of them on every test query are left out.

`bm25_char3`, `bm25_char4x`, `bm25_char5x`, `bm25_lyrmeta_nodiac12`, `bm25_meta`, `bm25_nodiac`, `bm25_nodiac12`, `bm25_phon12`, `bm25_syl`, `bm25_syl12`, `bm25_word`, `dense_naft_e5base_nonoise_s1`, `dense_naft_e5base_randneg_s1`, `dense_naft_e5base_s1`, `dense_naft_e5base_s1_rest`, `dense_naft_e5base_s2_rest`, `dense_naft_e5base_s3_rest`, `dense_naft_e5small_nonoise_s1`, `dense_naft_e5small_randneg_s1`, `dense_naft_e5small_s1`, `dense_naft_e5small_s1_rest`, `dense_naft_e5small_s2_rest`, `dense_naft_e5small_s3_rest`, `hybrid_naft_e5base_s3_rest_adaptive`, `hybrid_naft_e5base_s3_rest_equal`, `hybrid_naft_e5base_s3_rest_global`, `hybrid_naft_e5base_s3_rest_oracle`, `lexall_adaptive`, `lexall_equal`, `lexall_oracle`, `lexlyrics_equal`, `lextrain_adaptive`, `lextrain_equal`, `lextrain_global`, `lextrain_oracle`, `ql_syl`, `restoreall_bm25_syl`, `restoreall_bm25_syl12`, `restoretrain_bm25_syl`, `restoretrain_bm25_syl12`, `rm3_nodiac`, `rm3_syl`, `rm3_syl12`.

Left out as identical on every test query: `lexall_global` = `hybrid_naft_e5base_s3_rest_global`; `lexlyrics_adaptive` = `lexall_adaptive`; `lexlyrics_global` = `hybrid_naft_e5base_s3_rest_global`; `lexlyrics_oracle` = `lexall_oracle`.

C Tuned parameters

BM25 (k_1, b): syllables 0.3, 1.0; syllable uni+bigrams 0.3, 0.9; words 0.3, 1.0; unmarked syllables 0.6, 1.0; unmarked uni+bigrams 0.3, 0.75; character 3-grams 0.6, 1.0; character 4-grams across syllables 0.6, 0.9; character 5-grams across syllables 0.6, 0.9; sound-alike key 0.3, 0.9; metadata 0.9, 0.75. Query likelihood: $\mu = 5$. RM3: 10 feedback songs, 40 terms, original-query weight 0.5. Metadata weight: $w = 0.1$. Lexical RRF weights ($\text{Restore}_{\text{all}}$ uni+bigrams, unmarked uni+bigrams, character trigrams, metadata): (2, 0.5, 0, 0). All values were selected on the development split; the full list is in the released `results` files.

D Relation to the SEG301 and TMG301 courses

This appendix belongs to the course submission only and is removed from the journal version. The work is the joint project of the courses SEG301 (Search Engines) and TMG301 (Text Mining): SEG301 supplies the frame of a search engine (crawling, indexing, ranking, evaluation), TMG301 the text models inside it. Tables 15 and 16 map each lecture to the part of the work that applies it.

Table 8: Dense, fine-tuned, hybrid and reranked retrieval on the test split (MRR@10). Every choice of model or weights was made on the dev split.

	MRR@10 by query type						macro average		
System	Q1	Q2	Q3	Q4	Q5	Macro	S@1	S@10	R@100
queries	1241	1241	1241	227	227				
Reference									
RRF, weights tuned on dev	0.999	0.995	0.943	0.344	0.930	0.842	0.820	0.886	0.945
Dense, zero-shot (MaxP over 4-line passages)									
mE5-small				[not run yet]					
on restored queries				[not run yet]					
mE5-base				[not run yet]					
on restored queries				[not run yet]					
Vietnamese_Embedding				[not run yet]					
on restored queries				[not run yet]					
BGE-M3				[not run yet]					
on restored queries				[not run yet]					
bkai bi-encoder				[not run yet]					
on restored queries				[not run yet]					
Noise-aware fine-tuning (NA-FT)									
NA-FT mE5-base (3 seeds)	0.778‡	0.088‡	0.646‡	**0.702**†	0.639‡	0.571	0.516	0.691	0.818
on restored queries (3 seeds)	0.776‡	0.764‡	0.644‡	0.697†	0.646‡	0.705	0.646	0.831	0.937
NA-FT mE5-small (3 seeds)	0.782‡	0.121‡	0.628‡	0.551†	0.623‡	0.541	0.480	0.668	0.813
on restored queries (3 seeds)	0.779‡	0.768‡	0.628‡	0.540†	0.618‡	0.667	0.600	0.803	0.925
without noise (seed 1)	0.795‡	0.027‡	0.541‡	0.674†	0.700‡	0.547	0.493	0.658	0.760
without hard negatives (seed 1)	0.782‡	0.092‡	0.658‡	0.686†	0.672‡	0.578	0.524	0.699	0.827
without noise (seed 1)	0.792‡	0.056‡	0.568‡	0.542†	0.640‡	0.520	0.462	0.639	0.763
without hard negatives (seed 1)	0.779‡	0.130‡	0.635‡	0.551†	0.614‡	0.542	0.479	0.671	0.816
Hybrid: lexical lists + NA-FT mE5-base (seed 3), restored queries									
RRF, equal weights	0.859‡	0.862‡	0.790‡	0.434†	0.906	0.770	0.689	0.904	0.978
RRF, weights tuned on dev	**0.999**	**0.995**	**0.943**	0.344	**0.930**	0.842	0.820	0.886	0.945
RRF, weights per predicted type	0.998	**0.995**	**0.943**	0.532†	0.928	**0.879**	0.851	0.923	0.965
RRF, weights per true type (oracle)	0.999	0.995	0.943	0.674†	0.930	0.908	0.877	0.959	0.987
Cross-encoder reranking of the top 20 of the selected hybrid									
first stage, same queries	0.993	0.996	0.949	0.532†	0.928	0.880	0.852	0.924	0.964
bge-reranker-v2-m3	0.971‡	0.977‡	0.957	0.682†	0.895	0.896	0.870	0.931	0.964
Vietnamese_Reranker	0.945‡	0.943‡	0.914	0.650†	0.789‡	0.848	0.796	0.931	0.964

†/‡: significantly better/worse than the reference lexical system (RRF, weights tuned on dev; paired randomization test, Holm correction within each column, $p < 0.05$). Bold: best value per column among runs over every query, oracle rows excluded. NA-FT rows average the per-query scores of the seeds before testing. Reranking rows are evaluated on the queries that were reranked.

Table 9: One weight w for the dense list for every query: hybrid of the lexical lists (weights fixed at their tuned values) and NA-FT mE5-small (seed 3) on restored queries, MRR@10 on the test split. The w that maximises the macro average on the development split is 0.25. Last row: $w = 8$ for queries predicted as Q4 and Q4u, $w = 0$ otherwise.

w	Q1	Q2	Q3	Q4	Q5	Q4u	Macro
0	0.999	0.995	0.943	0.344	0.930	0.299	0.842
0.25	0.980	0.979	0.934	0.402	0.876	0.378	0.834
0.5	0.968	0.968	0.916	0.435	0.852	0.416	0.828
1	0.951	0.948	0.889	0.462	0.812	0.436	0.812
2	0.933	0.929	0.857	0.477	0.757	0.448	0.791
4	0.913	0.907	0.819	0.510	0.715	0.471	0.773
8	0.891	0.883	0.787	0.525	0.679	0.488	0.753
per predicted type	0.999	0.995	0.943	0.458	0.930	0.415	0.865

Table 10: Planned comparisons on the test split: change in MRR@10 from system A to system B (B − A), per query type and at 50% noise. *: $p < 0.05$, two-sided paired randomization test with 10,000 permutations, Holm correction over the 8 comparisons within each column. Systems with three seeds enter with their mean per query. Macro: change of the macro average, not tested. Reranking was run on 40 sampled queries per type.

Comparison (A → B)	Q1	Q2	Q3	Q4	Q5	N50	Macro
mE5-base: queries as typed → restored	0.000	+0.667*	+0.001	−0.001	+0.001	+0.008	+0.133
mE5-small, restored: zero-shot → NA-FT	+0.088*	+0.091*	+0.147*	+0.080*	−0.043	+0.140*	+0.073
NA-FT (seed 1) → without noise	−0.035*	−0.085*	−0.125*	+0.012	+0.059*	−0.172*	−0.035
NA-FT (seed 1) → random negatives	+0.004	−0.003	+0.005	+0.011	−0.004	+0.008	+0.002
Lexical fusion → hybrid	0.000	0.000	0.000	+0.114*	0.000	0.000	+0.023
Hybrid: predicted → true query type	0.000	0.000	0.000	+0.067*	0.000	0.000	+0.013
Hybrid: NA-FT mE5-small → mE5-base	−0.002	−0.001	0.000	+0.038*	−0.002	0.000	+0.006
Hybrid → cross-encoder reranking	−0.042	−0.025	−0.032	+0.104*	−0.080	–	−0.015

Table 11: Kinds of substituted syllables. Observed: 349 substitutions in 32,019 aligned syllables of 116 pairs of transcriptions of the same song, with 95% bootstrap intervals over pairs. N, E, W: realised shares in the noise sweep, classified the same way (all rates pooled). N draws four hand-set operations; E draws the same operations with the observed shares; W samples the observed kinds and substitution pairs.

	observed		sweep (%)		
Kind	n	% [95% CI]	N	E	W
Another word	164	47	0	0	46
Tone	63	18	15	18	18
Initial, listed confusion	9	3	15	12	2
Initial, other	36	10	10	11	13
Final consonant	3	1	10	3	1
Vowel mark	28	8	0	0	9
One letter (typing)	46	13	50	56	11

Table 12: Sensitivity of each query type over 43 systems: discriminative power (share of the 903 system pairs whose MRR@10 differs at $\alpha = 0.05$, two-sided randomization test) and Kendall's τ between the system ordering on the type and on the macro of the other main types, or, for the variants below the line, on the type it varies (in brackets).

Type	Queries	Disc. power (%)	τ
Q1	1241	84	0.40
Q2	1241	90	0.47
Q3	1241	86	0.57
Q4	227	79	-0.02
Q5	227	78	0.46
Q4u	227	80	0.32 (Q4)
Q5u	227	88	0.49 (Q5)
Q3u	1241	93	0.52 (Q3)
Q4s	199	76	0.90 (Q4)
Q5g	224	59	0.79 (Q5)

Table 13: MRR@10 on the test split when the index keeps a random share of the 6213 clusters (the target cluster is always kept; mean over 5 random draws). Rankings are those of the full index restricted to the kept songs; the hybrid is fused again from the restricted lists.

	Q1			Q4			Q5			Macro		
Share kept	25%	50%	100%	25%	50%	100%	25%	50%	100%	25%	50%	100%
BM25, syllables	0.912	0.877	0.840	0.440	0.383	0.325	0.868	0.834	0.802	0.604	0.561	0.521
BM25, unmarked bigrams	1.000	1.000	1.000	0.399	0.347	0.288	0.934	0.918	0.898	0.857	0.841	0.821
Restore + bigrams	1.000	0.999	0.999	0.459	0.401	0.342	0.956	0.945	0.932	0.876	0.859	0.840
Lexical RRF	1.000	0.999	0.999	0.459	0.399	0.344	0.954	0.944	0.930	0.877	0.859	0.842
mE5-base, restored	0.788	0.750	0.711	0.707	0.653	0.597	0.756	0.704	0.642	0.729	0.684	0.634
NA-FT (seed 1), restored	0.879	0.850	0.819	0.639	0.588	0.536	0.652	0.602	0.544	0.758	0.719	0.675
Selected hybrid	1.000	0.999	0.999	0.567	0.515	0.458	0.955	0.944	0.930	0.898	0.883	0.865

Table 14: Cost on four CPU cores without a GPU. Index: encoding the 79,328 passages (dense) or building the inverted index (lexical), and the size of the passage vectors. Query: encoding one query; search: exact search for one query (ms). mE5-small and NA-FT mE5-small share one architecture; their difference shows the variation between timing runs.

	Passages/s	Index	MB	Query (ms)	Search (ms)
mE5-small	83	16 min	122	6	5
NA-FT mE5-small	99	13 min	122	5	5
mE5-base	28	47 min	244	15	9
bkai bi-encoder	29	45 min	244	14	8
BM25, syllable uni+bigrams	–	3 s	–	–	0.3
BM25, unmarked uni+bigrams	–	4 s	–	–	0.4
Diacritic restoration	–	–	–	0.6	–
bge-reranker-v2-m3, top 20	–	–	–	–	100

Table 15: SEG301 lectures and where this work applies them.

Lecture	Applied as	Where
1 IR and search engines	known-item song search as the task	§3.4
2 Architecture	crawl, text transformation, index, query processing, ranking, evaluation; web demo	§3–4, code
3 Crawls and feeds	polite resumable crawler (robots.txt, 1 request/s), Unicode clean-up, near-duplicate clusters	§3.1, §3.2
4 Processing text	syllable, bigram, word, unmarked and character n-gram analysers; vocabulary statistics	§4.1
5 Ranking with indexes	sparse inverted index with precomputed BM25 weights, metadata field index	§4.1
6 Queries and interfaces	noisy-channel diacritic restoration, Telex decoding, RM3 expansion, result snippets	§4.2, code
7 Retrieval models	BM25, query likelihood, relevance model, dense bi-encoders, fine-tuning, cross-encoder	§4
8 Search engine optimisation	song pages with schema.org JSON-LD, sitemap, robots rules, click log (demo)	code (`app/`)
9 Evaluating search engines	test collection, MRR@10, paired randomization test with Holm correction, dev/test discipline, efficiency	§5, §6
10 Classification and clustering	query-type classifier; clusters of song versions	§4.3, §3.2
11 Social search	fusion of several result lists (metasearch, RRF)	§4.6
12 Beyond bag of words	term dependence through syllable bigrams; one search for all queries vs. type-adaptive fusion	§4.1, §4.6

Table 16: TMG301 lectures and where this work applies them (lectures 11–15 are not used).

Lecture	Applied as	Where
01 Regular expressions, normalisation, edit distance	lyric cleaning, NFC and diacritic handling, Damerau–Levenshtein distance of noisy queries	§3.1, §3.5
02 n-gram language models	Kneser–Ney trigram model for diacritic restoration	§4.2
03 Naive Bayes	query-type classification	§4.3
04 Logistic regression	class-balanced query-type classifier	§4.3
05 Vector semantics and embeddings	TF-IDF/BM25 weighting, dense vectors and cosine similarity	§4.1, §4.4
06 Clustering and text similarity	Jaccard similarity of shingles, MinHash, single-linkage clusters	§3.2
07 Transformers and pretrained models	zero-shot bi-encoders and cross-encoder reranking	§4.4, §4.7
08 Fine-tuning	noise-aware fine-tuning of a bi-encoder	§4.5
09 Machine translation (related)	diacritic restoration decoded like statistical translation (noisy channel, beam search)	§4.2
10 Chatbots and LLMs	LLM-written queries with automatic filters	§3.6
16 Word senses (related)	most-frequent-form baseline for restoration	§4.2